

MigrateFlow: approach, autonomy and next steps

Approach and implementation

I built MigrateFlow for an implementation consultant migrating inconsistent employee exports into a new platform. Multiple CSV/Excel files are profiled against a shared employee schema, mapped, cleaned, reconciled and validated before delivery to a mock API. React exposes live progress, exceptions, source provenance and per-record results; FastAPI and SQL persistence coordinate execution. Open-source Qwen2.5 through Ollama handles unfamiliar field meanings. My central design decision is that the model proposes semantics while deterministic code owns execution authority.

What the agent handles independently

Verified aliases with compatible types bypass inference. Remaining columns receive structured model proposals using masked profiling context, batched per file; saved proposals are reused on replay. A mapping is applied automatically only when its evidence score is at least 0.90, types are compatible and no blocking ambiguity or collision exists. This threshold is a conservative engineering rule, not a calibrated probability. Safe cleanup trims whitespace, normalizes known casing, converts dates with one interpretation and merges clear duplicates while retaining provenance. Every resulting record must pass validation.

Why and when I escalate

I draw the boundary where proceeding would require inventing a fact or choosing between plausible business meanings. Competing target fields, conflicting employee identities, ambiguous dates, missing required values and validation failures that safe cleanup cannot resolve require human judgment. For example, 03/04/2024 could mean 3 April or 4 March; confidence alone cannot settle the convention. Mapping collisions and model-recovery fallback also force review. Approval cannot bypass validation, and two source columns cannot silently overwrite one destination field. This avoids both unattended corruption and asking a consultant to confirm every routine transformation.

The review queue presents source context, a recommendation and the reason for escalation, with approve, correct or reject actions. Decisions persist across repeated processing. Resolving the final exception resumes the workflow. Selecting Autopilot authorizes sending valid records to the mock destination; guided mode retains a separate send decision. Both modes enforce the same safety policy.

Engineering beyond the model

A durable SQL job queue runs work outside browser requests, with bounded workers and renewable leases. Idempotent API writes, per-record outcomes, bounded transient retries, batch-scoped undo and an audit trail make actions inspectable and recoverable. Input limits and masked model context reduce resource and privacy risks. The hosted AWS prototype serves frontend, backend and local Ollama over HTTPS. Verification includes 62 backend tests, 13 frontend tests, a browser escalation/correction/delivery/undo walkthrough and real hosted Ollama inference. The hosted smoke took 123.5 seconds on CPU; this small test is not a general accuracy or throughput guarantee.

What I would build next

First, I would evaluate representative client exports and measure incorrect automatic decisions, review workload and completion time, then tune policy and inference using that evidence. Next come destination-specific authentication and rate limits, tenant isolation, encrypted storage, versioned migrations and stronger distributed recovery. The objective is demonstrably safer autonomy with less consultant effort, rather than adding unnecessary agent complexity.

Live demo: <https://demo-migrateflow.51-21-247-103.sslip.io>

Source repository: <https://github.com/sudharshanreddyragipindi35-collab/migrateflow-fde-assessment>